

Experiment 13 Robot Framework Browser Automation

Purpose

This experiment introduces the basic structure of a Robot Framework test. The test opens the SauceDemo website, confirms that the login button is present, and enters a sample username into the login form. It also shows how SeleniumLibrary turns readable test keywords into browser automation actions.

Learning goals

- Use Robot Framework sections, variables, test cases, and user-defined keywords.
- Import SeleniumLibrary and control a Chrome browser.
- Enter data into a web form with Input Text.
- Validate a page element with Page Should Contain Element.
- Run the test and inspect the generated HTML reports.

Tools and keywords

Item	Use
Robot Framework	Test automation framework
SeleniumLibrary	Browser and web element control
Open Browser	Starts a browser at a URL
Input Text	Writes text into a form field
Page Should Contain Element	Checks that a locator exists

Implementation

Create a folder, save the following content as experiment1.robot, and open the folder in VS Code.

```
*** Settings ***
Library      SeleniumLibrary
Suite Setup  Open Browser To Login Page
Suite Teardown  Close All Browsers

*** Variables ***
${URL}      https://www.saucedemo.com/
${BROWSER}  Chrome

*** Test Cases ***
Verify Login Page And Enter Username
    [Documentation]  Opens SauceDemo, checks the login button, and enters a username.
    Page Should Contain Element    id=login-button
    Input Text    id=user-name    standard_user

*** Keywords ***
Open Browser To Login Page
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
```

Installation and execution

1. Install Python 3.9 or later and Google Chrome.
2. In the project folder, create a virtual environment with `py -m venv .venv`.
3. Activate it with `.\venv\Scripts\Activate.ps1`.
4. Install packages using `python -m pip install -r requirements.txt`.
5. Run `robot --outputdir results experiment1.robot`.
6. Review `results/log.html` for detailed steps and `results/report.html` for the summary.

```
[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
outputs\Experiment 1 Rewritten.docx
```

Expected result

The test should open Chrome, load SauceDemo, find the login button, and place `standard_user` in the username field. The test should then close the browser and generate a passing report. Selenium Manager generally handles the compatible Chrome driver automatically.

Observation and conclusion

SeleniumLibrary provides a readable keyword layer over Selenium WebDriver. Because the test describes actions in plain language, the author does not need to write the lower-level browser commands directly. This keyword-driven approach makes small browser checks easier to read, maintain, and execute.